

9-1 Functions and Modular Programming in C++

There are two types of functions in C++:

1. **Standard Library Functions:** built-In functions in C++ (Predefined in C++).
2. **User-defined Functions:** Created by users.

In this lecture we explain the [user defined function](#).

✚ In C++, **functions are named blocks of code that perform specific tasks**, acting as the fundamental building blocks of **modular programming** (Modular programming is the process of subdividing a computer program into separate sub-programs), making the code easier to write, read, test, and maintain.

✚ C++ functions can take inputs (**called parameters**), execute a block of statements, and optionally **return a result**.

Function Syntax in C++

A function in C++ follows this general format:

```
return_type function_name(parameter_list)
{
    // body of the function
}
```

Each part has a specific role:

- **Return type:** Specifies what type of value the function returns. Use void if there is no return value.

- **Function name:** The name you will use to call the function.
- **Parameter list:** Inputs that the function accepts. It can be empty if no inputs are needed.
- **Function body:** The block of code that runs when the function is called.

9-2 Function Declaration vs Definition

 **Function Declaration** introduces the function to the compiler. It tells the compiler the return type, name, and parameters but does not include the body. It ends with a semicolon.

```
// Declaration  
int add(int, int);
```

Function Definition provides the actual implementation of the function.

```
int add(int a, int b) {  
    return a + b;  
}
```

Declaration is useful when the function is defined later in the code or in a separate file.

 *In simple terms, the **declaration** introduces the function to the compiler, while the **definition** provides the actual implementation.*

9-3 Calling a Function

Once a function is defined, you can use it by simply calling its name followed by parentheses. This tells the program to execute the code inside that function.

If the function takes parameters, you pass the required values inside the parentheses. If it doesn't take any parameters, you just use empty parentheses.

Example (1):

```
#include <iostream>
using namespace std;

// Void function definition
void hello() {
    cout << "computer engineering department" << endl;
}

// Return-type function definition
int square(int x) {
    return x * x;
}

int main() {
    // Calling the void function
    hello();
    int x;
    cout << "input value to x:: ";
    cin >> x;
    // Calling the return-type function
    int result = square(x);
    cout << "Square of x is: " << result << endl;
    return 0;
}
```

Output:

computer engineering department

input value to x:: 7

Square of x is: 49

9-4 Types of C++ User-defined Function According to Parameters (or Arguments) and Return Values

Type1: Function with no parameters and no return value.

Type 2: Function with parameters and no return value.

Type 3: Function with no parameters and with return value.

Type 4: Function with parameters and return value.

Example about type (1):

Write a c++ function named (info) to print University, College, Department then call it in the main program.

Sol.

```
#include <iostream>

using namespace std;

// Define the function info (This function with no arguments and no return value)
void info() {
    // Print the required statements
    cout << "University of Shatrah" << endl;
    cout << "College of Engineering" << endl;
    cout << "Department of Artificial Intelligence" << endl;
}

int main()
{
    // Call the info function
    info();
```

Output:

University of Shatrah

College of Engineering

Department of Artificial Intelligence

```
return 0;
}
```

Example about type (2):

```
#include <iostream>
```

```
using namespace std;
```

```
// Function that takes two integers as arguments and calculates their sum (but does not return it)
```

```
void sum(int num1, int num2)
```

```
{
```

```
    int result = num1 + num2;
```

```
    // Print the result
```

```
    cout << "The sum of the "<< num1<< " and " <<num2<<" is : " << result << endl;
```

```
}
```

```
int main()
```

```
{
```

```
int a, b;
```

```
// Ask the user to enter two integers
```

```
cout << "Enter the first number: ";
```

```
cin >> a;
```

```
cout << "Enter the second number: ";
```

```
cin >> b;
```

```
// Call the sum function with user inputs
```

```
sum(a, b);
```

```
// Call the sum function with predefined values
```

Output:

Enter the first number: 12

Enter the second number: 13

The sum of the 12 and 13 is : 25

The sum of the 10 and 90 is : 100

```
sum(10, 90);  
return 0;  
}
```

Example about type (3):

Return Values

The void keyword, used in the previous examples, indicates that the function should not return a value. If you want the function to return a value, you can use a **data type** (such as int, float, char, string, etc.) instead of void, and **use the return keyword** inside the function: see the following example

Write a c++ function named (mul) which returns the product of two numbers (v1=5 and v2=10) to the main program.

Sol.

```
#include <iostream>  
  
using namespace std;  
  
// Define the function mul which returns the product of two numbers  
// (v1=5 and v2=10)  
int mul()  
{  
    int v1 = 5;  
    int v2 = 10;  
  
    // Return the product of the two values v1 and v2  
    return v1 * v2;  
}
```

Output :

The product of the two values is: 50

```
int main()
{
// Call the mul function and store the returned value
int res = mul();
// Print the result res
cout << "The product of the two values is: " << res << endl;
return 0;
}
```

Example about type (4):

Write a C++ function that calculate the sum of two integers and returns the result to the main program.

Sol.

```
#include <iostream>
using namespace std;
// Function that returns the sum of two integers
int sum(int num1, int num2)
{
// Return the sum of num1 and num2
return num1 + num2;
}
int main()
{
```

```
int a, b;
// Ask the user to enter two integers
cout << "Enter the first number: ";
cin >> a;
cout << "Enter the second number: ";
cin >> b;
// Call the sum function and store the result
int result = sum(a, b);
// Print the result
cout << "The sum of the two numbers is: " << result << endl;
int result2= sum(90,10);
cout << "The sum of the two numbers is: " << result2 << endl;
return 0;
}
```

Output:

Enter the first number: 5

Enter the second number: 10

The sum of the two numbers is: 15

The sum of the two numbers is: 100